

Streaming and Adaptive Bit Rate: the QoE of Video Delivery

440-2216/01 Quality of Service • Lecture 07

Jan Rozhon • Department of Telecommunications, FEECS

Where we are going

Part 1

HTTP adaptive streaming

Why video moved from RTP to HTTP; ladder, segments, manifest

↔ Lecture 06

Part 2

The playout buffer

Buffer dynamics, startup delay, stalling

↔ Lecture 02 queues

Part 3

ABR algorithms

Throughput-, buffer-based, hybrid, learned

new

Part 4

QoE and the network

Stalls, switches, startup; QoS
→ QoE mapping

↔ Lectures 03, 05, 09

One question runs through the lecture: **when the network cannot deliver the bit rate the viewer wants, what should the player sacrifice — picture quality, smoothness, or continuity?**

What you should be able to do

1. Explain why video streaming moved from RTP/UDP to HTTP over TCP and describe the ladder-segment-manifest structure of DASH and HLS.
2. Write the buffer balance of a player, compute startup delay and time to stall for a given bit rate and throughput.
3. Explain how throughput-based, buffer-based, hybrid (MPC), and learned ABR algorithms choose the next segment, and name each one's typical failure.
4. Rank the QoE factors of adaptive streaming — startup delay, stalling, quality level, switching — and evaluate a simple QoE model.
5. Map network QoS parameters (throughput, its variation, RTT, loss) onto streaming QoE, and name mechanisms by which the network can assist the player.

Keep asking: **is this impairment caused by the network, by the player's decision, or by the encoding?**

In adaptive streaming, packet loss becomes waiting

TCP hides every lost packet — the viewer sees stalls, startup delay and quality changes instead of artefacts

Part 1

From RTP to HTTP adaptive streaming



Why streaming moved to HTTP

RTP/UDP streaming (RTSP, 1990s–2000s)

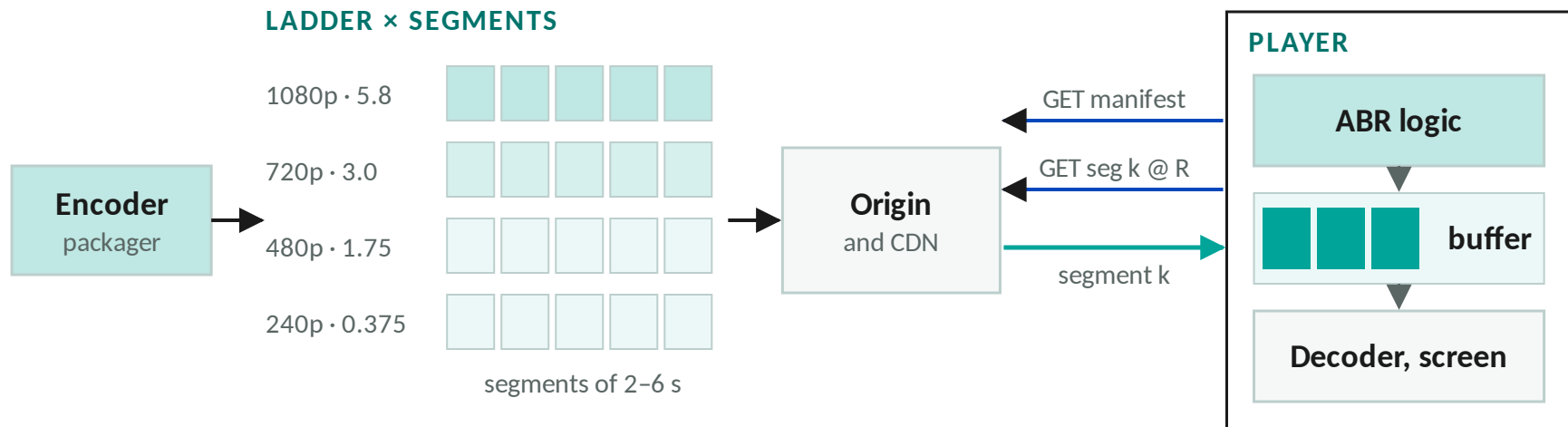
- server keeps per-client session state
- packet loss visible as artefacts
- blocked by firewalls and NAT
- needs dedicated streaming servers
- low latency — still used for conversational video (WebRTC)

HTTP adaptive streaming (2008 →)

- stateless web servers, cacheable by any **CDN**
- TCP (or QUIC) retransmits losses — no artefacts
- passes every firewall on port 443
- the **client** adapts the bit rate per segment
- latency of seconds, not milliseconds

Today HTTP adaptive streaming carries the majority of all Internet traffic in volume; RTP remains the choice where end-to-end delay matters more than scale.

How HTTP adaptive streaming works



The server is a plain web server; every adaptation decision is made by the client, one segment at a time.

The **ladder** offers each title at several resolution-bit-rate pairs; each version is cut into **segments** of 2–6 s that start with an I-frame, so the player can switch rung at every segment boundary.

The standards

Standard	Body, year	Role
MPEG-DASH	ISO/IEC 23009-1, 2012	codec-agnostic; manifest = XML <i>Media Presentation Description</i> (MPD)
HLS	Apple 2009; RFC 8216, 2017	manifest = M3U8 playlists; required on Apple devices
CMAF	ISO/IEC 23000-19, 2018	common fragmented-MP4 segment format for both DASH and HLS
SAND	ISO/IEC 23009-5, 2017	messages between player and network elements
CMCD	CTA-5004, 2020	player reports buffer, bit rate, and throughput to the CDN in each request
LL-HLS, LL-DASH	2019–2020	low-latency live: chunked CMAF, partial segments

The standards fix formats and messages — **not** the adaptation algorithm. Every player may implement its own ABR logic.

Checkpoint — where did the packet loss go?

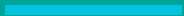
A link used for two services loses 1 % of its packets. The VoIP call from Lab 03 sounds clearly degraded. A HAS video on the same link shows no blocking or smearing at all. Why — and does that mean the loss is harmless for the video?

Predict first · discuss with a neighbour · then reveal

TCP retransmits every lost segment, so every video byte eventually arrives intact — no artefacts. But loss **reduces TCP throughput** (congestion window halving) and adds retransmission delay. The player sees a slower link and reacts: it chooses a lower bit rate or, if the buffer runs out, stalls. The loss has changed form, not disappeared.

Part 2

The playout buffer



The buffer balance

The buffer B holds seconds of video. While a segment of duration τ and bit rate R downloads over throughput C :

$$\frac{dB}{dt} = \frac{C}{R} - 1$$

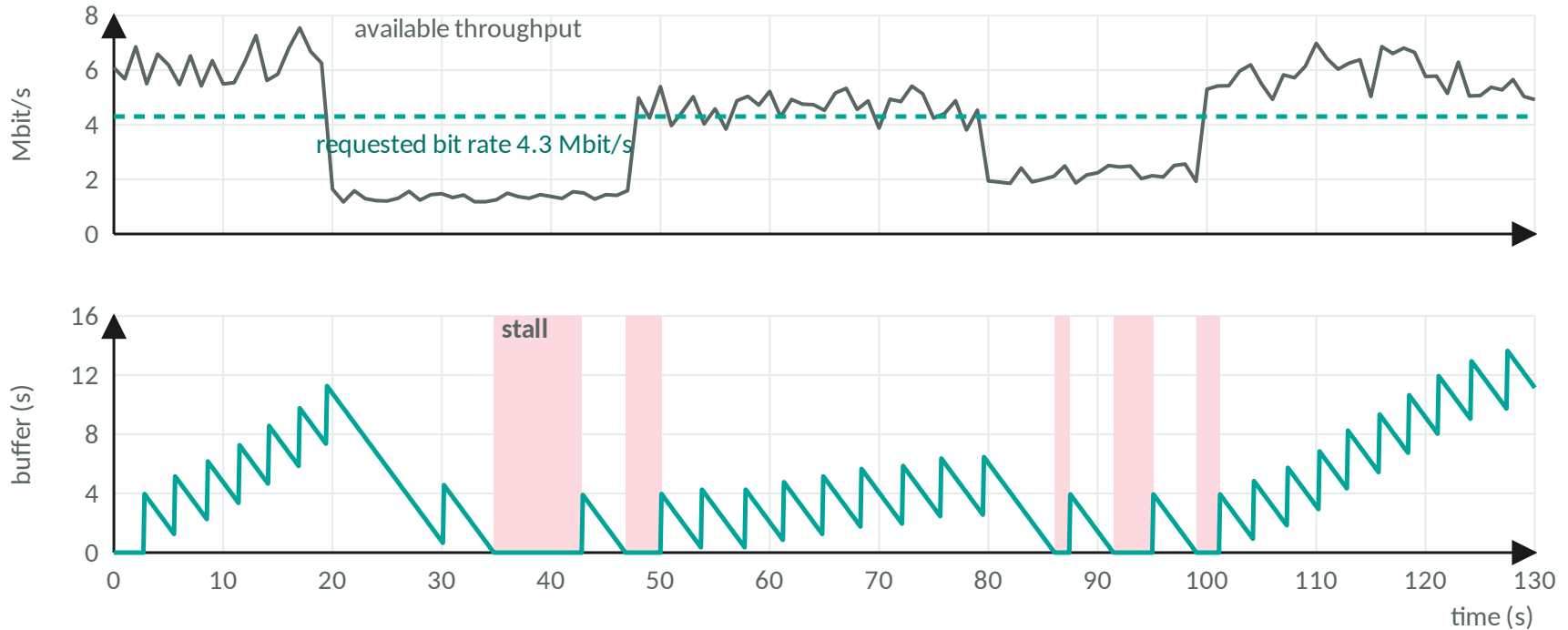
- each second of download adds C/R seconds of video,
- playback removes one second per second,
- $B = 0$ during playback $\rightarrow$ **stall** (rebuffering).

The sustainability condition

$R \leq C$ on average — the same condition as $\rho < 1$ for a queue in Lecture 02.

Download time of one segment: $T_k = \frac{R_k \tau}{C_k}$. The player sees only these T_k , not C directly.

A player that never adapts



Each 4 s segment takes $4.3/C \times 4$ s to download: whenever $C < 4.3$ Mbit/s the buffer drains — here 5 stalls, 18 s in total.

Worked example — startup and time to stall

Startup delay (first segment, $\tau = 4$ s, $C = 6.5$ Mbit/s, ignoring RTTs):

First rung	Download
0.375 Mbit/s	$1.5/6.5 = 0.23$ s
4.3 Mbit/s	$17.2/6.5 = 2.65$ s

Time to stall at $R = 3$ Mbit/s after the link drops to $C = 2$ Mbit/s with $B = 12$ s buffered:

$$\frac{dB}{dt} = \frac{2}{3} - 1 = -\frac{1}{3} \Rightarrow t_{\text{stall}} = 36 \text{ s}$$

36 s of warning — enough for any algorithm that watches the buffer to step down before the viewer notices.

Checkpoint — how big should the buffer be?

A video-on-demand player allows 60 s of buffer; a live sports stream is watched 6 s behind the live edge. What does the larger buffer buy, and why can the live player not simply use it too?

Predict first · discuss with a neighbour · then reveal

A large buffer **absorbs throughput dips** — at $C/R = 0.5$, 60 s of buffer survive two minutes of degraded link. But the buffer cannot hold video that does not exist yet: a live player can be at most as far ahead as it is **behind the live edge**. Low latency and robustness to throughput variation are in direct conflict; low-latency live streaming (2–6 s) is the hardest case for ABR.

Part 3

Adaptive bit rate algorithms

The ABR problem

Before each segment k the player chooses a rung R_k from the ladder, knowing only the past download times and the current buffer.

Its goal is conflicting:

- **high** bit rate → better picture,
- **few** switches → smooth experience,
- **no** stalls, **short** startup.

A common objective (Yin et al., 2015) — the **linear QoE**:

$$\text{QoE} = \sum_k q(R_k) - \lambda \sum_k |q(R_{k+1}) - q(R_k)| \\ - \mu T_{\text{stall}} - \mu_s T_s$$

$q(\cdot)$ quality of a rung (often $q(R) = R$), T_{stall} total stall time, T_s startup delay, λ, μ, μ_s weights.

Throughput-based adaptation

Estimate the throughput from recent segments and pick the highest rung below it, with a safety margin:

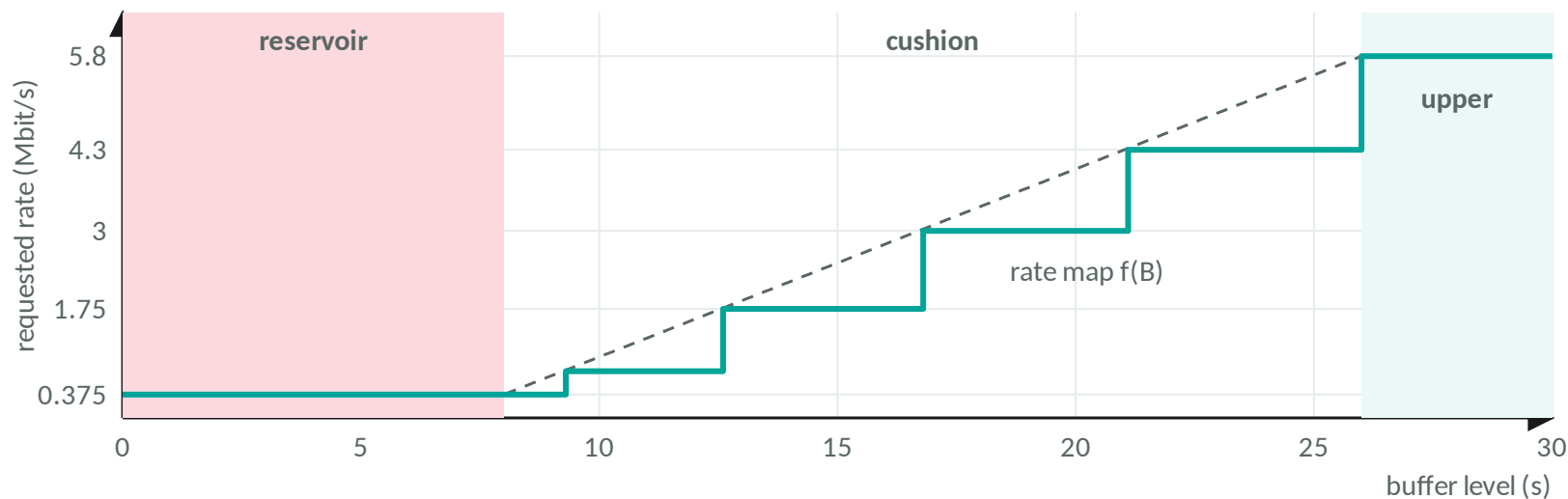
$$\hat{C} = \frac{n}{\sum_{i=1}^n 1/C_{k-i}}, \quad R_k = \max\{R \leq \alpha \hat{C}\}$$

The **harmonic mean** damps outliers (FESTIVE, Jiang et al., 2012); $\alpha \approx 0.8-0.9$.

Failure modes

- reacts **late** to a sudden drop: the estimate is still high when the buffer is already low
- **ON-OFF** download pattern: while the buffer is full the player idles, and short downloads underestimate TCP's capacity
- **downward spiral** next to a competing TCP flow: lower rate $\rightarrow$ smaller segments $\rightarrow$ lower estimate (Huang et al., IMC 2012)
- **unfairness** between players sharing a bottleneck

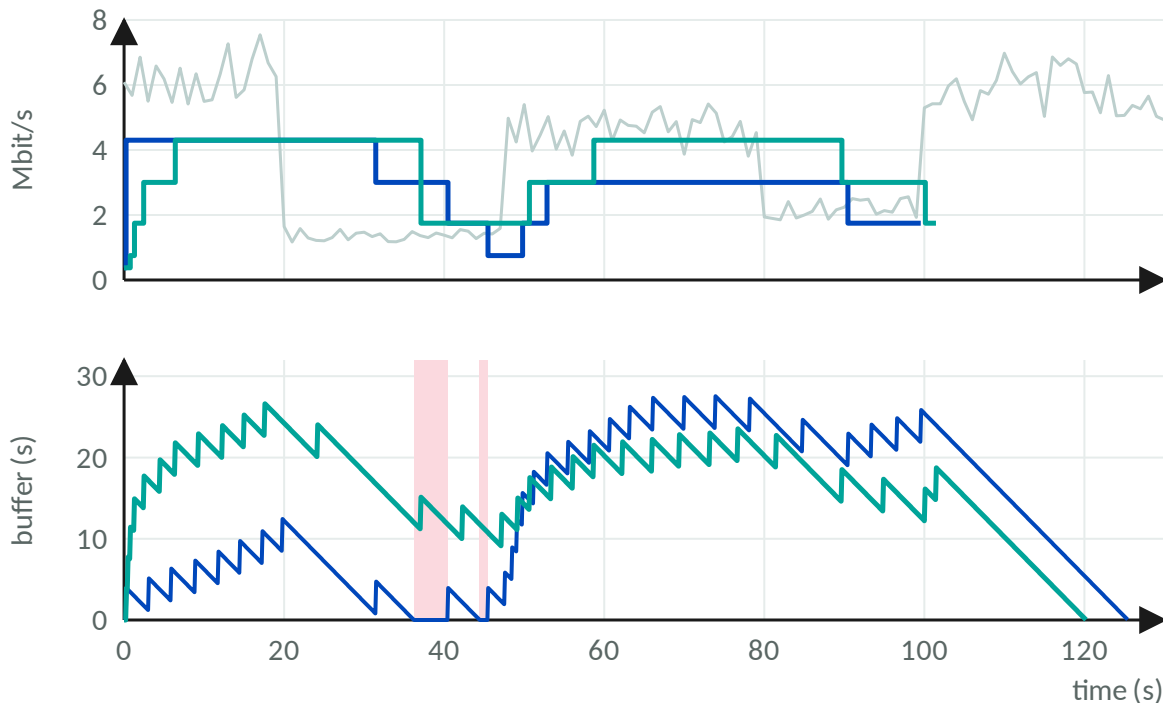
Buffer-based adaptation



The buffer level alone chooses the rate: no throughput estimate is needed while the buffer is in the cushion.

BBA (Huang et al., SIGCOMM 2014, Netflix trial): choose the rate from the **buffer level alone**. The reservoir protects against stalls; the cushion trades buffer for quality. Throughput estimation is needed only at startup.

Two algorithms, one link



— throughput-based
— buffer-based (BBA-0)
— available throughput

SESSION METRICS

	rate	buffer
mean bit rate, Mbit/s	2.71	2.98
bit-rate switches	7	9
stalls	2	0
stall time, s	5.2	0.0
linear QoE	39	76

Same link, same ladder: the rule that picks the next segment decides whether the viewer ever sees a stall.

Hybrid and learned algorithms

Model predictive control

MPC (Yin et al., SIGCOMM 2015): predict throughput for the next few segments, optimize the linear QoE over that horizon, apply only the first decision, repeat.

Hybrid rules

dash.js *DYNAMIC*: throughput rule at startup and after seeks, BOLA once the buffer is established — the best of both failure modes.

Learned policies

Pensieve (Mao et al., SIGCOMM 2017): reinforcement learning trained on throughput traces. Fugu (Yan et al., NSDI 2020): learned transmit-time predictor inside MPC.

Stanford's **Puffer** study streamed live TV to real viewers for a year: in the wild, simple buffer-based control matched several sophisticated schemes, and only an algorithm trained *in situ* improved on it.

Checkpoint — which algorithm, when?

A viewer's phone moves from Wi-Fi to a congested cell: throughput falls from 20 to 1.5 Mbit/s in one second. The player has 25 s of buffer at 4.3 Mbit/s. Which reacts better — throughput-based or buffer-based — and what does each do in the next 20 s?

Predict first · discuss with a neighbour · then reveal

The **throughput-based** player's next estimate still averages the fast Wi-Fi samples, so it requests another 4.3 Mbit/s segment; its download takes 11.5 s, only then does the estimate collapse and it jumps to the lowest rung — one abrupt switch. The **buffer-based** player also finishes that segment, but its rate map lowers the request step by step as the buffer falls through the cushion. With 25 s of buffer neither stalls; the buffer-based one degrades more gracefully.

Part 4

Streaming QoE and the network

The QoE factors of adaptive streaming

Stalling — number, duration, and position of rebuffering events. The **dominant** factor.

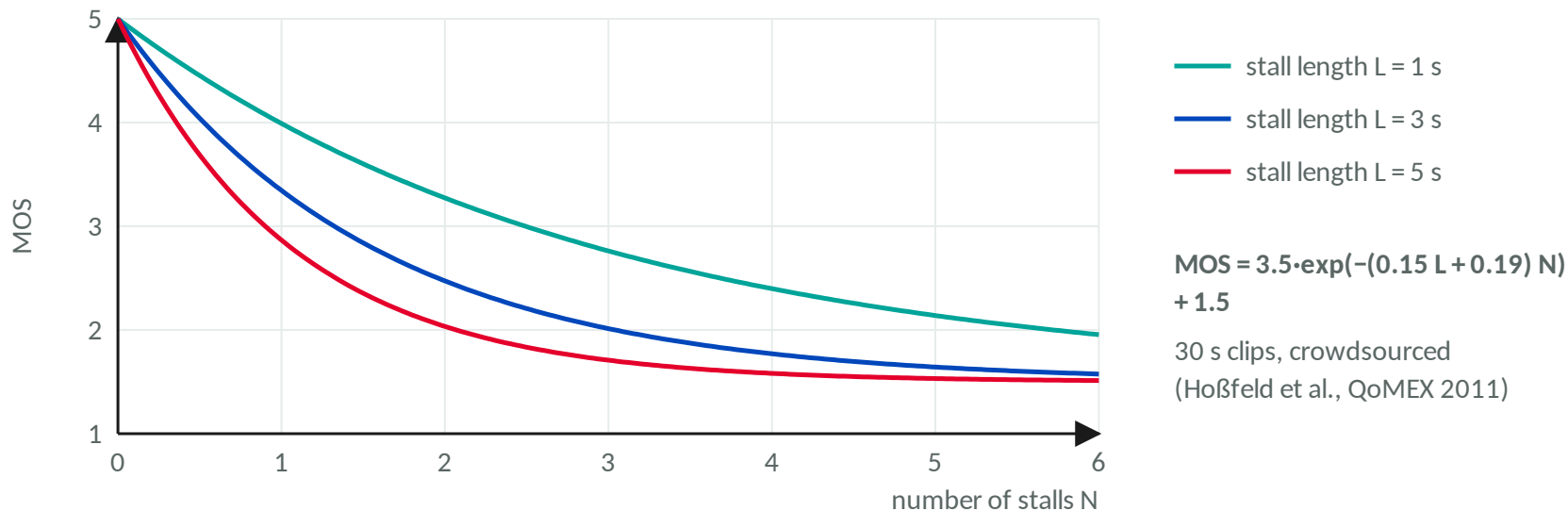
Initial (startup) delay — tolerated much better than a stall of the same length: viewers expect waiting at the start.

Quality level — bit rate, resolution, and the resulting visual quality (Lecture 06 metrics).

Quality switching — frequency and amplitude; downward and abrupt switches hurt most, gradual steps are preferred.

Ordering from the literature (Seufert et al., 2015): **stalling** $\gg$ **initial delay**, and **stalling** $>$ **low but constant quality** $>$ **frequent switching**.

How much does a stall cost?



Number of stalls matters more than their length: a single 1 s stall already costs about one MOS point.

One 3 s stall in a 30 s clip: $3.5 e^{-(0.15 \cdot 3 + 0.19)} + 1.5 = 3.35$ — from "excellent" to "fair" with a single event.

Viewers vote with their feet

2 s

Viewers begin to abandon a video whose startup delay exceeds 2 s; each further second adds about **5.8 %** abandonment (Krishnan & Sitaraman, IMC 2012, 23 M views).

1 % → 3 min

A 1 % increase of the **buffering ratio** (stall time / session time) cut viewing time of a live event by more than 3 minutes (Dobrian et al., SIGCOMM 2011).

Engagement — abandonment, viewing time, return rate — is QoE measured at scale without asking anyone. It is what streaming providers actually optimize.

Worked example — two sessions, one score

Five 4 s segments; linear QoE with $q(R) = R$, $\lambda = 1$, $\mu = 4.3$:

	Session A	Session B
Rates, Mbit/s	3, 3, 3, 3, 3	4.3, 4.3, 1.75, 4.3, 4.3
$\sum q$	15.0	18.95
Switching	0	$2.55 + 2.55 = 5.1$
Stall	0 s	$2 \text{ s} \rightarrow 8.6$

A: 15.0 • B: 5.25

B delivered **26 % more bits** and scores **less than half** of A. A constant middle rung beats an ambitious one that must fall back — the ranking of the factors slide, in numbers.

From network QoS to streaming QoE

Network QoS parameter	Effect in the player	QoE factor
Mean throughput	highest sustainable rung	quality level
Throughput variation	estimate errors, buffer drain	switching, stalls
Round-trip time	slower TCP ramp-up, request gaps	startup delay, lower throughput
Packet loss	retransmission, cwnd halving → lower throughput	quality level, stalls — not artefacts
Jitter	absorbed by the buffer	none, if the buffer is non-empty

TCP throughput under loss (Mathis et al., 1997): $C \approx \frac{\text{MSS}}{\text{RTT}} \cdot \frac{1.22}{\sqrt{p}}$ — with MSS 1460 B, RTT 50 ms, $p = 0.1\%$: $C \approx 9$ Mbit/s per connection.

The network can help

Tell the network

CMCD: the player adds its buffer level, bit rate, and measured throughput to every request; the CDN can prioritize players about to stall.

Tell the player

SAND and **CMSD**: network or CDN elements send hints — available bandwidth, cache state — so the player does not have to guess.

Control the network

SDN (Lecture 09): a controller that sees all players on a bottleneck allocates bandwidth or queues per player for fairness and fewer stalls.

Network assistance closes the loop between QoS mechanisms and QoE — at the price of cooperation between parties that encrypted HTTPS traffic otherwise keeps apart.

Checkpoint — start delay or stall?

A provider can either (a) start every video 3 s later with a larger initial buffer, or (b) start in 0.5 s and accept one 2 s stall in 10 % of sessions. Which improves QoE, and which improves engagement?

Predict first · discuss with a neighbour · then reveal

For **MOS**, stalls cost far more than initial delay, so (a) looks safer. For **engagement**, 3 s is past the 2 s abandonment threshold: every session pays it, and $\approx 6\%$ of viewers leave before the video starts — while (b) hurts only one session in ten. The answer depends on which QoE measure the provider optimizes — and a good ABR (small startup rung, buffer-based ramp-up) avoids having to choose.

Common mix-ups

Bit rate vs throughput

Bit rate is what the player requests;
throughput is what the network delivers.
Stalls come from the gap.

Stall vs startup delay

Both are waiting, but viewers penalize a
mid-stream stall far more.

Standard vs algorithm

DASH and HLS define formats; the ABR
algorithm is each player's own choice.

Loss vs artefacts

Over TCP, loss causes lower throughput
and stalls, not blocking.

More bits vs better QoE

A higher mean bit rate with stalls or
switches can score lower than a constant
lower rate.

MOS vs engagement

Ratings and viewer behaviour are both
QoE measures — and can disagree.

Summary

HTTP adaptive streaming Ladder $\times$ segments $\times$ manifest on plain web servers and CDNs; the client adapts.

Buffer balanced $B/dt = C/R - 1$; stall when $B = 0$; sustainable only if $R \leq C$.

Throughput-based ABR Estimates C from the past; late on sudden drops, confused by ON-OFF traffic.

Buffer-based ABR Rate from buffer level; robust against stalls, slower to ramp up.

Hybrid and learned ABR MPC optimizes a QoE objective over a horizon; learned schemes must be tested in the wild.

QoE factors Stalling $\gg$ initial delay; constant quality beats frequent switching.

QoS $\rightarrow$ QoE mapping Loss and RTT act through TCP throughput; jitter vanishes in the buffer.

Network assistance CMCD, SAND, and SDN let the network and the player cooperate.

Exit questions — use the ideas

1. A player requests 3 Mbit/s segments of 4 s over a 2.4 Mbit/s link with 10 s buffered. When does it stall if it never switches?
2. Why does a throughput-based player that idles because its buffer is full tend to underestimate the available throughput?
3. Session A: 1 stall of 6 s. Session B: 3 stalls of 2 s. Using the exponential model, which scores higher, and why?
4. An operator cuts packet loss on a path from 0.4 % to 0.1 %. Using the Mathis formula, by what factor can per-connection throughput rise, and which QoE factors could improve?

Explain the mechanism, not just its name.

Where you use this

There is no dedicated streaming lab yet; the tools of the other labs cover the pieces:

- `tc` shaping and `netem` (Lab 03, Lab 05) create the throughput traces of Part 2
- PSNR, SSIM, VMAF (Lab 04) score the rungs of the ladder
- the `simpy` pipeline (Lab 02) can model the buffer as a queue

Try it on your own machine: limit your link, open the dash.js reference player, and watch its buffer and bit-rate graphs.

```
sudo tc qdisc add dev eth0 root \  
    tbf rate 2mbit burst 32kbit \  
    latency 400ms  
# play in the dash.js reference player  
sudo tc qdisc del dev eth0 root
```

References — standards

- ISO/IEC 23009-1, *Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats*, 5th ed., 2022.
- R. Pantos, W. May, "HTTP Live Streaming," RFC 8216, 2017.
- ISO/IEC 23000-19, *Common media application format (CMAF) for segmented media*, 2018.
- ISO/IEC 23009-5, *DASH — Part 5: Server and network assisted DASH (SAND)*, 2017.
- CTA-5004, *Web Application Video Ecosystem — Common Media Client Data*, 2020.
- ITU-T P.1203, *Parametric bitstream-based quality assessment of progressive download and adaptive audiovisual streaming services over reliable transport*, 2017.

References — literature

- M. Seufert et al., "A survey on quality of experience of HTTP adaptive streaming," *IEEE Communications Surveys & Tutorials*, 17(1), 469–492, 2015.
- T. Hoßfeld et al., "Quantification of YouTube QoE via crowdsourcing," *IEEE ISM Workshop on Multimedia Quality*, 2011.
- F. Dobrian et al., "Understanding the impact of video quality on user engagement," *ACM SIGCOMM*, 2011.
- S. S. Krishnan, R. K. Sitaraman, "Video stream quality impacts viewer behavior," *ACM IMC*, 2012.
- T.-Y. Huang et al., "Confused, timid, and unstable: picking a video streaming rate is hard," *ACM IMC*, 2012.
- T.-Y. Huang et al., "A buffer-based approach to rate adaptation: evidence from a large video streaming service," *ACM SIGCOMM*, 2014.
- X. Yin et al., "A control-theoretic approach for dynamic adaptive video streaming over HTTP," *ACM SIGCOMM*, 2015.
- K. Spiteri, R. Urgaonkar, R. K. Sitaraman, "BOLA: near-optimal bitrate adaptation for online videos," *IEEE INFOCOM*, 2016.
- H. Mao, R. Netravali, M. Alizadeh, "Neural adaptive video streaming with Pensieve," *ACM SIGCOMM*, 2017.
- F. Y. Yan et al., "Learning in situ: a randomized experiment in video streaming," *USENIX NSDI*, 2020.
- M. Mathis et al., "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM CCR*, 27(3), 1997.

Thank you for your attention

Jan Rozhon

+420 596 995 900 · jan.rozhon@vsb.cz

www.vsb.cz